

BITS, PILANI – K. K. BIRLA GOA CAMPUS

Theory of Computation

(CS F51)

Lecture No. 6



Finite Representation of Languages

Languages are often infinite and so a natural question arises if a **finite representation** is possible?

$L = \{w \in \{0, 1\}^* : w \text{ has two or three occurrences of } 1 \text{ the first and the second of which are not consecutive} \}$

The language can be represented as

$$L = \{0\}^*\{1\}\{0\}^*\{0\} \circ \{1\}\{0\}^*(\{1\}\{0\}^* \cup \emptyset^*)$$

We will dispense with the symbols braces and $\circ$

$$L = 0^*10^*010^*(10^* \cup \emptyset^*)$$

We call expression above (and others alike) a **regular expression**

Finite Representation of Languages

$$\Sigma = \{0, 1\}$$

1. $L = \{w : \text{the first and last symbols of } w \text{ are equal}\}$

The language can be represented as

$$0(0 \cup 1)^*0 \cup 1(0 \cup 1)^*1 \cup 0 \cup 1$$

2. $L = \{w : \text{every odd position in } w \text{ is } 1\}$

An **even-length** valid string must have a 1 at every odd index, followed by either 0 or 1 at the subsequent even index. This forms recurring pairs: 11, 10, 11 etc.

An **odd-length** valid string follows the exact same pattern, but terminates with a final trailing 1 at its last odd position.

$$(1(0 \cup 1))^*(1 \cup \phi^*)$$

Regular Expression Definition

Definition

We define a $\mathcal{R}$ of **regular expressions** over an alphabet Σ as follows

$\mathcal{R} \subseteq (\Sigma \cup \{ (,), \emptyset, \cup, * \})^*$ and $\mathcal{R}$ is the smallest set such that

1. $\emptyset \in \mathcal{R}$ and $\Sigma \subseteq \mathcal{R}$, i.e. we have that

$$\emptyset \in \mathcal{R} \text{ and } \forall_{\sigma \in \Sigma} (\sigma \in \mathcal{R})$$

2. If $\alpha, \beta \in \mathcal{R}$, then

$(\alpha\beta) \in \mathcal{R}$ **concatenation**

$(\alpha \cup \beta) \in \mathcal{R}$ **union**

$\alpha^* \in \mathcal{R}$ **Kleene's Star**

Regular Expressions

Definition

The function $\mathcal{L} : \mathcal{R} \longrightarrow 2^{\Sigma^*}$ is defined recursively as follows

1. $\mathcal{L}(\emptyset) = \emptyset, \quad \mathcal{L}(\sigma) = \{\sigma\} \quad \text{for all } \sigma \in \Sigma$

2. If $\alpha, \beta \in \mathcal{R}$, then

$$\mathcal{L}(\alpha\beta) = \mathcal{L}(\alpha) \circ \mathcal{L}(\beta) \quad \text{concatenation}$$

$$\mathcal{L}(\alpha \cup \beta) = \mathcal{L}(\alpha) \cup \mathcal{L}(\beta) \quad \text{union}$$

$$\mathcal{L}(\alpha^*) = \mathcal{L}(\alpha)^* \quad \text{Kleene's Star}$$

Regular Languages

Definition

A language $L \subseteq \Sigma^*$ is *regular*

if and only if

L is represented by a *regular expression*,

i.e. there is $\alpha \in R$ such that $L = L(\alpha)$

Questions regarding Regular Languages

Natural Questions:

1. Is *Union/Intersection/Complement* of regular languages regular? **Union Yes**, just by the definition of regular expression
2. Is *Concatenation/Kleene Star* of regular languages regular? **Yes**, just by the definition of regular expression
3. *How many* regular languages are there?
Or *Are all languages regular*?
4. *If not*, which language is not regular and how to show that language is not regular?

Questions regarding Regular Languages

Answer to Question 3

Remember

Theorem

The set $\mathcal{R}$ of **regular expressions** over an alphabet Σ is **countably infinite**

A language $L \subseteq \Sigma^*$ is **regular** if and only if L is represented by a **regular expression**

Therefore, the set of regular languages on Σ is countably infinite

Whereas set of all languages on Σ is uncountably infinite.

So **uncountably infinite languages are not regular.**

Finite State Automaton

We have looked at regular languages

Next, we study mathematical model of machines that can recognize regular languages.

Informally, these languages can be “*processed*” by computers having a very small amount of memory.

Before we give a formal definition of a *finite automaton*, we consider an example in which such an automaton shows up in a natural way.

An example: Controlling a toll gate

We consider the problem of designing a *“computer”* that controls a toll gate.

When a car arrives at the toll gate, the gate is closed.

The gate opens as soon as the driver has payed 25 cents.

We assume that we have only three coin denominations: 5, 10, and 25 cents.

For simplicity, we also assume that no excess change is returned.

Controlling a toll gate

After having arrived at the toll gate, the driver inserts a sequence of coins into the machine.

At any moment, the machine has to decide whether or not to open the gate, i.e., whether or not the driver has paid 25 cents (or more).

In order to decide this, the machine is in one of the following six states, at any moment during the process:

- The machine is in state (*initial state*) q_0 , if it has not collected any money yet.
- The machine is in state q_1 , if it has collected exactly 5 cents.

Controlling a toll gate

- The machine is in state q_2 , if it has collected exactly 10 cents.
- The machine is in state q_3 if it has collected exactly 15 cents.
- The machine is in state q_4 , if it has collected exactly 20 cents.
- The machine is in state (*final/accepting state*) q_5 , if it has collected 25 cents or more.

Initially (when a car arrives at the toll gate), the machine is in state q_0 .

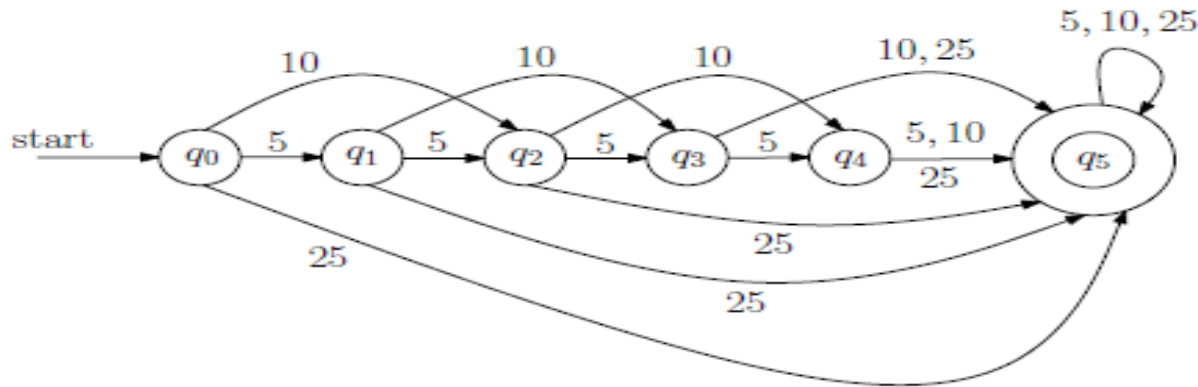
Assume, for example, that the driver presents the sequence (10,5,5,10) of coins.

Controlling a toll gate

- After receiving the first 10 cents coin, the machine switches from state q_0 to state q_2 .
- After receiving the first 5 cents coin, the machine switches from state q_2 to state q_3 .
- After receiving the second 5 cents coin, the machine switches from state q_3 to state q_4 .
- After receiving the second 10 cents coin, the machine switches from state q_4 to state q_5 .
- At this moment, the gate opens.
- Remember that no change is given back

Controlling a toll gate

We can represent *pictorially* the behavior of the machine for all possible sequences of coins.



State q_5 is represented by *two circles*, because it is a special state: As soon as the machine reaches this state, the gate opens.

State q_0 has an incoming arrow, indicating it is the *start state*

Controlling a toll gate

Observe that the machine (or computer) only has to remember which state it is in at any given time.

Thus, it needs only a very small amount of memory